



《实用Python编程》

第9讲：利用Scikit-learn进行机器学习

实用Python编程 教研组



数据智能

- 数据价值的释放依赖以机器学习特别是深度学习为代表的AI技术

数据智能旨在自动从数据中提取关键特征，帮忙我们在特定任务场景下做出更好的决策

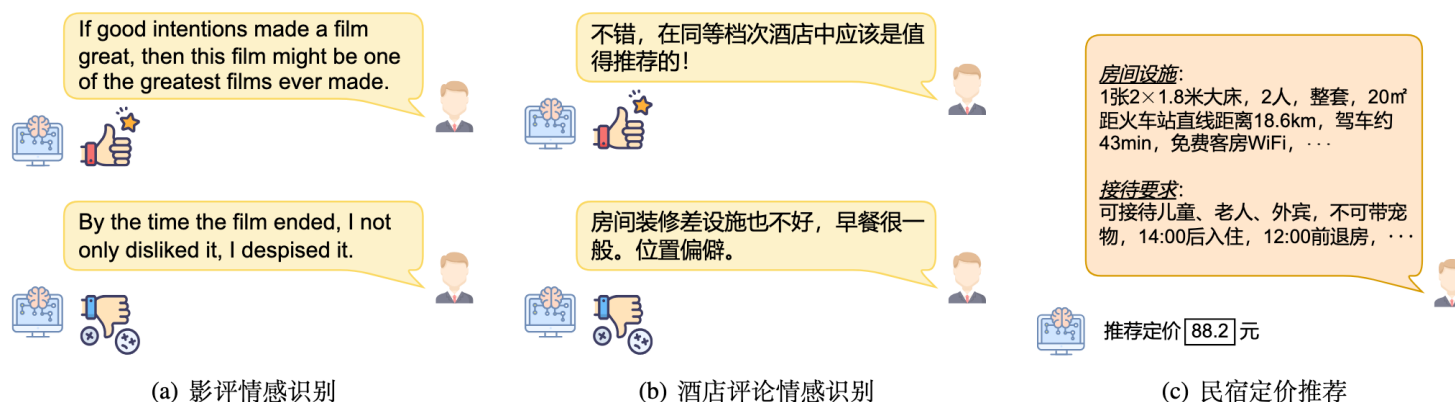
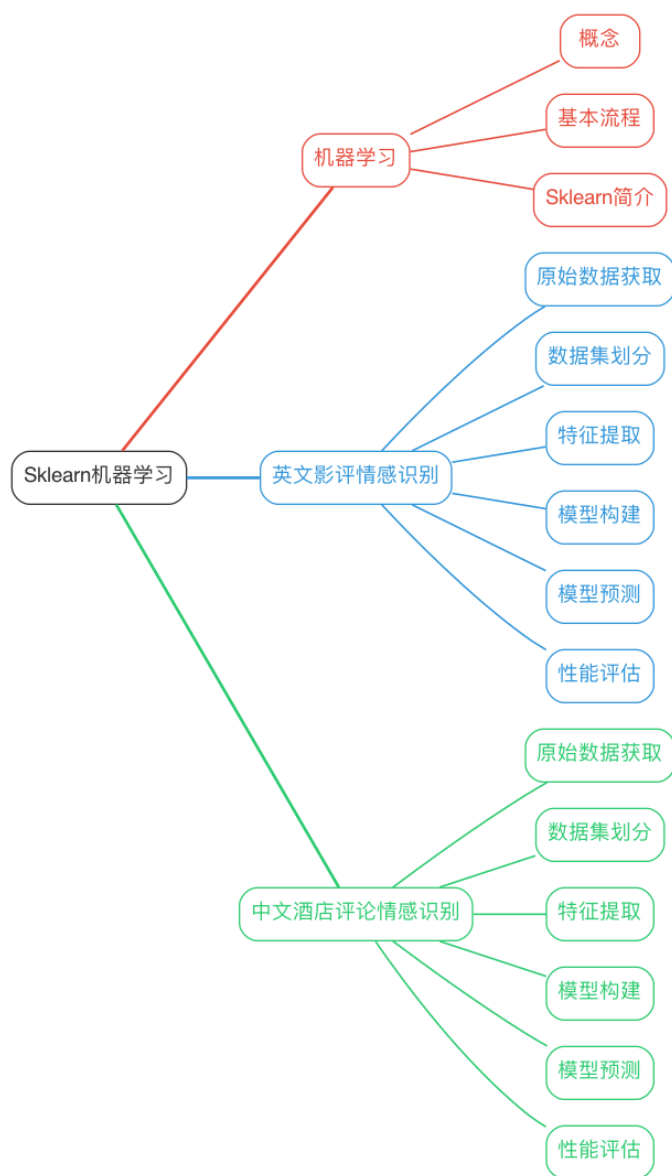


图 5.1: 数据智能案例展示

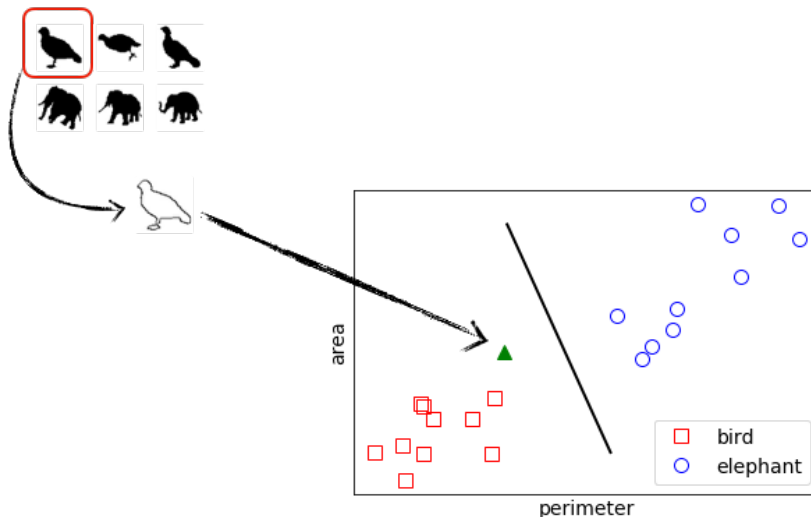
酒店评论情感识别在帮助消费者“避雷”的同时，也帮助经营者定位需要改进的薄弱环节

提纲



机器学习简介

- 机器学习 (machine learning)
 - 计算机在特定算法的指导下, 利用已知数据得出适当的模型, 并基于此模型对新的情境给出判断的过程



训练阶段:

从样本中提取特征 (比如物体的面积、周长)

- 实际任务中特征往往是高维度的
- 在特征空间寻找可将不同类别区分的决策边界
- 边界由相应参数控制 (比如直线的斜率、截距)

测试阶段:

算法根据新样本 (图中三角) 相对边界的位置判定其类别

机器学习简介

- 了解基本流程，建立宏观视角 (overall picture)
 - 两阶段：先训练，后测试

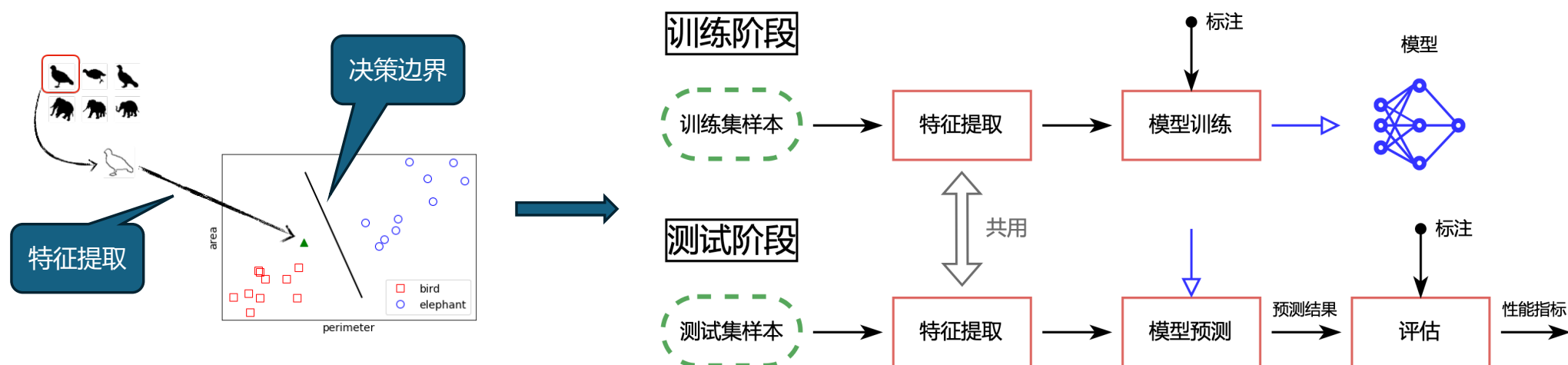


图 5.2: 机器学习典型的两阶段流程

注意观察左图视觉元素与右图各功能模块的对应关系

从概念到实践

- 利用Scikit-learn进行机器学习
 - Scikit-learn (sklearn) : 功能齐全的Python机器学习库
 - 官网 <https://scikit-learn.org/>
 - 安装指令 `pip install scikit-learn`
 - 导入方式 `import sklearn`

scikit-learn

Machine Learning in Python

[Getting Started](#) [Release Highlights for 1.9](#)

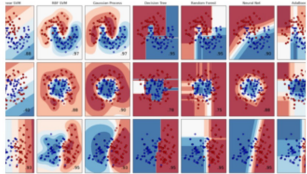
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)

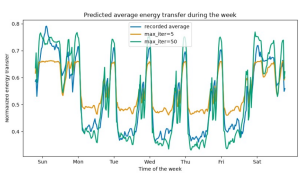


Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)

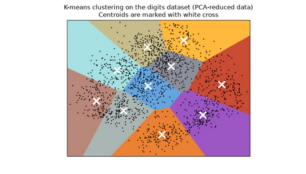


Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



从概念到实践

• 基于sklearn进行机器学习的主要步骤

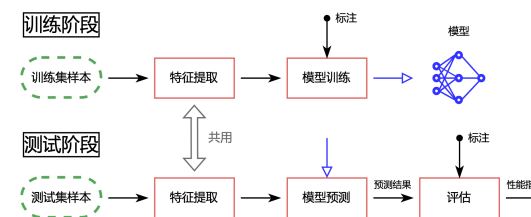
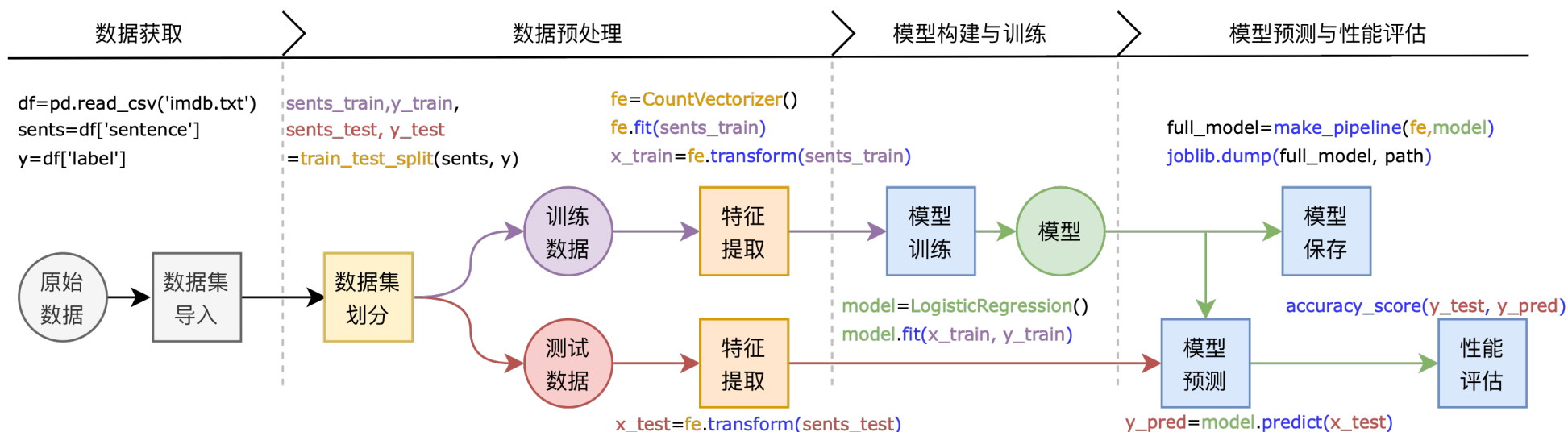


图 5.2: 机器学习典型的两阶段流程



imdb.txt: 影评数据

fe: 特征提取器

model: 逻辑回归模型

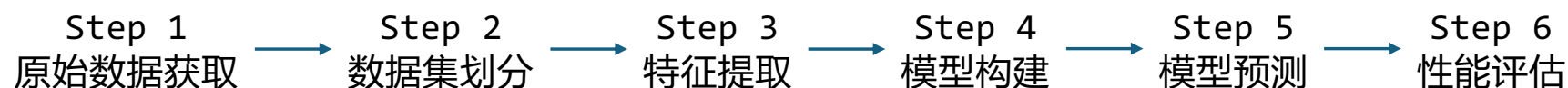
案例1: 英文影评情感识别

- 搭建一个机器学习模型自动判断一个短影评的情感
 - 正面 (positive) : 好评
 - 负面 (negative) : 差评

*This is the kind of money
that is wasted properly.*

*By the time the film ended, I not
only disliked it, I despised it.*

这是一个二分类任务 (binary classification)



案例1: 英文影评情感识别

- Step 1. 原始数据获取
 - imdb_labeled.txt: 带有情感标注的英文影评数据集
 - 每行存储一个样本, 由两个字段组成
 - 第1列: 影评文本
 - 第2列: 情感标签 (0表示负面, 1表示正面)

jupyter imdb_labeled.txt Last Checkpoint: 9 months ago

File Edit View Settings Help

```
1 A very, very, very slow-moving, aimless movie about a distressed, drifting young man.    0
2 Not sure who was more lost - the flat characters or the audience, nearly half of whom walked out.    0
3 Attempting artiness with black & white and clever camera angles, the movie disappointed - became even more ridiculous - as the acting was poor
  and the plot and lines almost non-existent.    0
4 Very little music or anything to speak of.    0
5 The best scene in the movie was when Gerardo is trying to find a song that keeps running through his head.    1
6 The rest of the movie lacks art, charm, meaning... If it's about emptiness, it works I guess because it's empty.    0
```



案例1: 英文影评情感识别

- Step 1. 原始数据获取
 - 使用Pandas.read_csv读取
 - 默认会将引号作为字段的限定符, 因此需设置quoting参数为csv.QUOTE_NONE

```
1 import pandas as pd
2 import csv
3
4 data_file = 'pybook-data/ch5/imdb_labelled.txt' #指定数据文件路径
5 df = pd.read_csv(data_file, names=["sentence", "label"], sep="\t", quoting=csv.QUOTE_NONE)
6 print(df.shape) # 输出 (1000, 2)
7 sents = df['sentence'].values #读取影评
8 y = df['label'].values #读取每条影评对应的二值标签
```

	sentence	label
0	A very, very, very slow-moving, aimless movie ...	0
1	Not sure who was more lost - the flat characte...	0
2	Attempting artiness with black & white and cle...	0
3	Very little music or anything to speak of.	0
4	The best scene in the movie was when Gerardo i...	1
...	...	...

案例1: 英文影评情感识别

- Step 2. 数据集划分
 - 将整个数据集按8:2比例划分为训练集和测试集, 并保存

```
1 import os
2 from sklearn.model_selection import train_test_split
3
4 out_dir = 'output'
5 train_file = f'{out_dir}/imdb_labelled_train.csv'
6 test_file = f'{out_dir}/imdb_labelled_test.csv'
7 if os.path.exists(train_file) and os.path.exists(test_file): #若文件已存在, 直接导入
8     print(f'Files exist at "{train_file}" and "{test_file}"')
9     df_train, df_test = pd.read_csv(train_file), pd.read_csv(test_file)
10    sents_train, sents_test = df_train['sentence'].values, df_test['sentence'].values
11    y_train, y_test = df_train['label'].values, df_test['label'].values
12 else:
13     os.makedirs(out_dir, exist_ok=True)
14     sents_train, sents_test, y_train, y_test = train_test_split(sents, y, test_size=0.2, random_state=1)
15     print(f'训练样本数量: {len(sents_train)}') # 800
16     print(f'测试样本数量: {len(sents_test)}') # 200
17     # 将划分后的数据保存到文件, 以便重复使用该划分方案
18     df_train = pd.DataFrame({'sentence': sents_train, 'label': y_train})
19     df_test = pd.DataFrame({'sentence': sents_test, 'label': y_test})
20     # 保存为CSV文件
21     df_train.to_csv(train_file, index=False)
22     df_test.to_csv(test_file, index=False)
```

训练样本数量: 800
测试样本数量: 200



案例1: 英文影评情感识别

- Step 3. 特征提取
 - 词袋特征 (bag-of-words, BoW)

BoW特征的每一维对应一个词

	all	embarrassing	insult	intelligence	one	word
All in all its an insult to one's intelligence	2	0	1	1	1	0
In a word, it is embarrassing	0	1	0	0	0	1

停止词 (stopwords) : a, an, in, is等过于常见, 缺乏特征表示能力, 会被忽略

案例1: 英文影评情感识别

- Step 3. 特征提取
 - 使用sklearn中的CountVectorizer提取BoW特征

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 # 使用训练数据构建BoW特征提取器
3 fe = CountVectorizer()
4 fe.fit(sents_train)
5 vob = fe.get_feature_names_out()
6 print(f'词表大小: {len(vob)}') # 词表大小: 2647
7 print(f'部分词汇: {vob[-5:]}') # 部分词汇: ['youthful' 'youtube' 'yun' 'zillion' 'zombiez']
8 # 根据词表提取BoW特征
9 x_train = fe.transform(sents_train)
10 x_test = fe.transform(sents_test)
11 print(f'训练集特征规模: {x_train.shape}') # 训练集特征规模: (800, 2647)
12 print(f'测试集特征规模: {x_test.shape}') # 测试集特征规模: (200, 2647)
```

根据定义，测试集代表“未来”，在训练阶段不可见，因此只能在训练集上拟合特征提取器

查看特征

```
1 i = 2
2 print(df_train['sentence'][i]) # It's a sad movie, but very good.
3 values = x_train[i].data
4 columns = x_train[i].indices
5 print(vob[columns], values)
```

```
It's a sad movie, but very good.
['but' 'good' 'it' 'movie' 'sad' 'very'] [1 1 1 1 1 1]
```

案例1: 英文影评情感识别

- Step 4. 模型构建
 - 训练一个逻辑回归分类器 (LRC)并保存, 以便复用

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.pipeline import make_pipeline
3 import joblib
4
5 clf = LogisticRegression() # 构建模型
6 clf.fit(x_train, y_train) # 训练模型
7 full_model = make_pipeline(fe, clf) # 封装特征提取器与分类器
8 joblib.dump(full_model, 'movie_review_bow_lrc.joblib') # 保存为一个本地文件
```

案例1: 英文影评情感识别

- Step 5. 模型预测
 - 使用LRC在已提取BoW特征的测试集上预测

```
1 y_pred = clf.predict(x_test)
2 probs = clf.predict_proba(x_test)
3 print(y_pred.shape, probs.shape)
```

(200,) (200, 2)

表 5.2: Sklearn 模型的预测方法 (n 表示测试样本数, C 表示类别数)

方法	决策类型	返回值
<code>predict</code>	硬决策, 给出样本所属类别	一个尺寸为 $(n,)$ 的 <code>ndarray</code> 对象
<code>predict_proba</code>	软决策, 给出样本属于每个类别的概率值	一个尺寸为 (n, C) 的 <code>ndarray</code> 对象

在本案例中, 类别数 $C=2$

案例1: 英文影评情感识别

- Step 5. 模型预测
 - 使用DataFrame观察分类结果

```
1 import numpy as np
2
3 df_test['negative'] = probs[:,0]
4 df_test['positive'] = probs[:,1]
5 df_test['argmax'] = np.argmax(probs, axis=1)
6 df_test['y_pred'] = y_pred
7 df_test
```

probs的第1列 (索引0)、第2列 (索引1)
分别是测试样本属于负类、正类的得分

argmax返回数组每一行中最大值所在列的索引

	sentence	label	negative	positive	argmax	y_pred
0	By the time the film ended, I not only dislike...	0	0.823776	0.176224	0	0
1	The acting, as you'd expect from this cast, is...	1	0.536713	0.463287	0	0
2	The plot doesn't hang together at all, and the...	0	0.746511	0.253489	0	0
3	I felt as though her going to Ireland did abso...	0	0.454886	0.545114	1	1
4	If good intentions made a film great, then thi...	1	0.037038	0.962962	1	1
...	...	...	...	...	...	...

新添加的列

得分高的类胜出

案例1: 英文影评情感识别

- Step 6. 性能评估
 - 定性观察: 找几个例子试一试

```
1 full_model = joblib.load('movie_review_bow_lrc.joblib')
2
3 input_texts = ['this is a good movie', 'the movie is horrible', 'a horrible movie', 'not a bad movie']
4 prob = full_model.predict_proba(input_texts)
5 pred = np.argmax(prob, axis=1)
6 names = ['负面评论', '正面评论']
7
8 for i, text in enumerate(input_texts):
9     j = pred[i]
10    print(f'{text} -> {names[j]} ({prob[i][j]:.2f})')
```

this is a good movie -> 正面评论 (0.66)
the movie is horrible -> 负面评论 (0.68)
a horrible movie -> 负面评论 (0.68)
not a bad movie -> 负面评论 (0.97)

案例1：英文影评情感识别

- Step 6. 性能评估

- 定性观察：LRC找出的主要特征是否符合常识

```
1 vob = fe.get_feature_names_out()
2 weights = clf.coef_.ravel()
3 word_weight_table = pd.DataFrame({'单词':vob, 'LRC系数': weights})
4 sorted_table = word_weight_table.sort_values(by='LRC系数', ascending=False)
```

单词 LRC系数			单词 LRC系数		
0	10	0.929517	2602	wonderful	1.421288
1	12	0.107174	1015	great	1.337117
2	13	-0.097498	213	beautiful	1.286196
3	15pm	-0.110906	1395	love	1.251405
4	18th	0.094482	1353	liked	1.247918
...	...	...	...	...	...
2642	youthful	-0.206285	2201	stupid	-1.149934
2643	youtube	-0.425369	2621	would	-1.204261
2644	yun	-0.098880	1559	no	-1.345176
2645	zillion	-0.206397	182	awful	-1.390163
2646	zombiez	-0.165176	189	bad	-2.310350

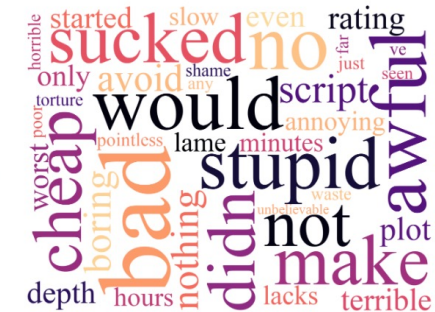
(a) 按单词排序

(b) 按权重排序

6.3节 WordCloud



(a) 权重较大的词汇



(b) 权重较小的词汇

图 5.6: LRC 提取的影评正面/负面词汇

图 5.5: 影评词汇与 LRC 情感分类器决策权重对照

案例1: 英文影评情感识别

- Step 6. 性能评估
 - 定量评价: 基于混淆矩阵的指标计算

	实际阳性 Np	实际阴性 Nn	
预测为阳性	TP	FP (误报)	Precision = TP / (TP + FP)
预测为阴性	FN (漏报)	TN	

TPR / Sensitivity / Recall = TP / (TP + FN)

Accuracy = (TP+TN)/(Np+Nn)

FPR = FP / (TN + FP)
Specificity = 1 - FPR

理想的混淆矩阵是什么样的?

案例1: 英文影评情感识别

- Step 6. 性能评估
 - 定量评价: 基于混淆矩阵的指标计算

```
1 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
2
3 accuracy = accuracy_score(y_test, y_pred)
4 print(f'分类准确率: {accuracy:.3f}')
5 cm = confusion_matrix(y_test, y_pred)
6 tn, fp, fn, tp = cm.ravel()
7 report = classification_report(y_test, y_pred, digits=3)
8 print(report)
```

分类准确率: 0.815

	precision	recall	f1-score	support
0	0.840	0.800	0.820	105
1	0.790	0.832	0.810	95
accuracy			0.815	200
macro avg	0.815	0.816	0.815	200
weighted avg	0.816	0.815	0.815	200

测试集中每个类别
的实际样本数

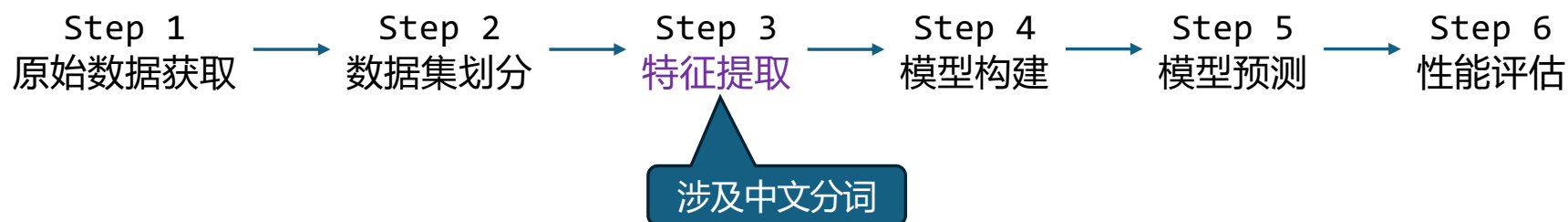
案例2: 中文酒店评论情感识别

- 任务形态与案例1类似，只不过数据从英文变成了中文

酒店的环境还是挺不错的，一大片的高尔夫球场

房间比较脏，尤其是地毯，像80年代的东西。最令人难以忍受的是早餐，由于有旅行团，人比较多

这也是一个二分类任务，沿用案例1的流程



案例2: 中文酒店评论情感识别

- Step 2. 数据集划分
 - 将整个数据集按8:2比例划分为训练集和测试集, 并保存

```
1 out_dir = 'output'
2 train_file = f'{out_dir}/hotel_labelled_train.csv'
3 test_file = f'{out_dir}/hotel_labelled_test.csv'
4 if os.path.exists(train_file) and os.path.exists(test_file): #若文件已存在, 直接导入
5     print(f'Files exist at "{train_file}" and "{test_file}"')
6     df_train, df_test = pd.read_csv(train_file), pd.read_csv(test_file)
7     sents_train, sents_test = df_train['review'].values, df_test['review'].values
8     y_train, y_test = df_train['label'].values, df_test['label'].values
9 else:
10     os.makedirs(out_dir, exist_ok=True)
11     sents = df['review'].values
12     y = df['label'].values
13     sents_train, sents_test, y_train, y_test = train_test_split(sents, y, test_size=0.2, random_state=1)
14     # 将划分后的数据保存到文件, 以便重复使用该划分方案
15     df_train = pd.DataFrame({'label': y_train, 'review': sents_train})
16     df_test = pd.DataFrame({'label': y_test, 'review': sents_test})
17     # 保存为CSV文件
18     df_train.to_csv(train_file, index=False)
19     df_test.to_csv(test_file, index=False)
20 print(f'训练样本数量: {len(sents_train)}') # 训练样本数量: 6212
21 print(f'测试样本数量: {len(sents_test)}') # 测试样本数量: 1553
```

训练样本数量: 6212
测试样本数量: 1553

案例2: 中文酒店评论情感识别

• Step 3. 特征提取

- 考虑到中文特有的分词需求, 通过tokenizer参数指定中文分词函数
- 相比案例1, 案例2的样本数和特征维度 (26k vs. 2.6k) 大一个数量级

```
1 import jieba
2
3 jieba.initialize()
4 def jieba_tokenize(text):
5     return jieba.lcut(text) # 把一段中文文本切分成词语列表
6
7 fe = CountVectorizer(tokenizer=jieba_tokenize)
8 fe.fit(sents_train)
9 vob = fe.get_feature_names_out()
10 print('词表大小:', len(vob)) # 词表大小: 26262
11 print('部分特征:', vob[10000:10005]) # 部分特征: ['安贞桥' '安逸' '安阳' '安静' '安静下来']
12
13 x_train = fe.transform(sents_train)
14 x_test = fe.transform(sents_test)
15
16 print(f'训练集特征规模: {x_train.shape}') # 训练集特征规模: (6212, 26262)
17 print(f'测试集特征规模: {x_test.shape}') # 测试集特征规模: (1553, 26262)
```

案例2: 中文酒店评论情感识别

- Step 4. 模型构建
 - 训练一个LRC
 - 样本数与特征维度变得更大, 需将LRC的迭代轮次参数max_iter (默认值为 100) 设置为1000

```
1 clf = LogisticRegression(max_iter=1000)
2 clf.fit(x_train, y_train)
3 # 以下为可选操作
4 full_model = make_pipeline(fe, clf)
5 joblib.dump(full_model, 'hotel_review_bow_lrc.joblib')
```

案例2: 中文酒店评论情感识别

- Step 5. 模型预测 + Step 6. 性能评估
 - 定性观察: 找几个例子试一试

```
1 full_model = joblib.load('hotel_review_bow_lrc.joblib')
2
3 input_texts = ['房间干净整洁, 服务态度超好.',
4                '位置很方便, 早餐也很丰盛.',
5                '隔音很差, 一晚上没睡好',
6                '房间有异味, 前台态度冷漠.'],
7
8 prob = full_model.predict_proba(input_texts)
9 pred = np.argmax(prob, axis=1)
10 names = ['负面评论', '正面评论']
11
12 for i, text in enumerate(input_texts):
13     j = pred[i]
14     print(f'{text} -> {names[j]} ({prob[i][1]:.2f})')
```

} AI生成

房间干净整洁, 服务态度超好。 -> 正面评论 (0.79)
位置很方便, 早餐也很丰盛。 -> 正面评论 (0.93)
隔音很差, 一晚上没睡好 -> 负面评论 (0.26)
房间有异味, 前台态度冷漠。 -> 负面评论 (0.34)

案例2: 中文酒店评论情感识别

- Step 5. 模型预测 + Step 6. 性能评估
 - 定量评价: 基于混淆矩阵的指标计算

```
1 y_pred = clf.predict(x_test)
2 accuracy = accuracy_score(y_test, y_pred)
3 print (f'分类准确率: {accuracy:.3f}')
```

```
4 cm = confusion_matrix(y_test, y_pred)
5 tn, fp, fn, tp = cm.ravel()
6 report = classification_report(y_test, y_pred, digits=3)
7 print(report)
```

分类准确率: 0.875

	precision	recall	f1-score	support
0	0.807	0.779	0.793	476
1	0.904	0.917	0.911	1077
accuracy			0.875	1553
macro avg	0.855	0.848	0.852	1553
weighted avg	0.874	0.875	0.874	1553